

For a dense layer with n_{in} inputs and n_{out} neurons, the number of trainable weights and biases is

$$N_{\text{param}}^{(\ell)} = n_{\text{in}}n_{\text{out}} + n_{\text{out}}. \quad (3.10)$$

Increasing width or depth can improve the representational capacity, but it also increases parameter storage, arithmetic effort, and the risk of fitting dataset-specific details. Validation data, regularisation, and a suitable architecture search are therefore required to balance approximation capability and generalisation.

Figure 3.2 summarizes this feedforward structure and links the scalar neuron calculation in Equation (3.7) to the layer-wise mapping in Equation (3.8). Each layer transforms the complete activation vector of its predecessor. The grey connections denote full connectivity, while the highlighted edge from input x_i to hidden neuron j identifies the scalar matrix entry $w_{ji}^{(1)} = [\mathbf{W}^{(1)}]_{ji}$. Information propagates only from the input toward the output.

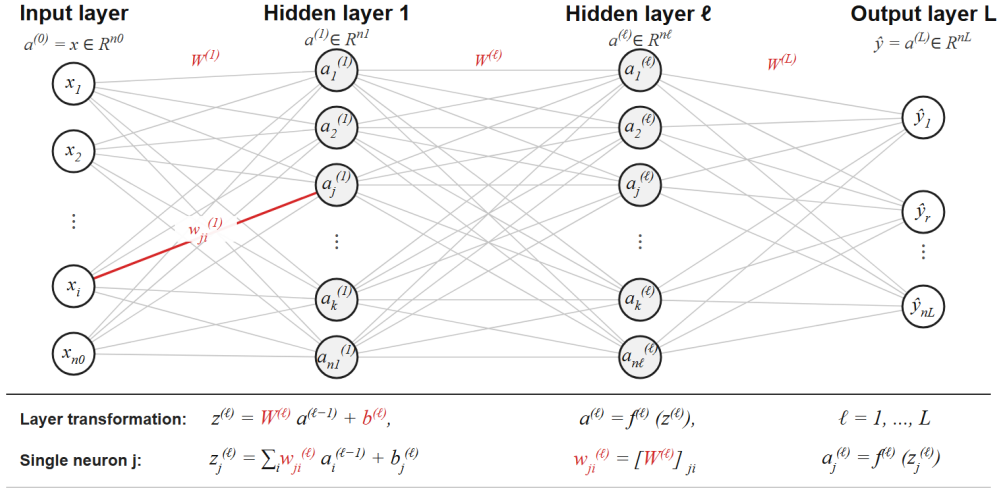


Figure 3.2: General structure and layer-wise mathematical mapping of a fully connected multilayer perceptron. The input vector $\mathbf{x} = \mathbf{a}^{(0)}$ is propagated through hidden representations $\mathbf{a}^{(\ell)}$ to the output $\hat{\mathbf{y}} = \mathbf{a}^{(L)}$. The highlighted connection from neuron i to neuron j represents the scalar entry $w_{ji}^{(1)}$ of the matrix $\mathbf{W}^{(1)}$. The bias is added separately at the target neuron.

An MLP applies the same static mapping to every input vector and has no internal temporal state. Time dependence must therefore be supplied explicitly, for example through cumulative quantities or a lag sequence of previous measurements. This gives the designer direct control over the represented history and allows efficient parallel training. The limitation is that the selected history length and